

SHOPMIND

Technical Document — Architecture, Implementation & Failure Handling

Track 1: AI Shopping Agent | Kasparro Agentic Commerce Hackathon | April 2026

What this document is

The technical companion to the Product Document. It answers: how did we build it, how do the components connect, where does AI end and deterministic code begin, what happens when things break, and what are the known limitations. Read alongside the Product Document for the full picture.

T1. System Architecture

T1.1 High-level component map

ShopMind consists of four distinct layers. The boundary between the AI layer and the deterministic layer is the most important architectural decision in the system — it determines what can fail gracefully and what cannot.

Layer	Components	Responsibility
1. Interface layer	React frontend, mood canvas UI, chat thread, confidence score display, cart widget	Renders conversation, captures buyer input, displays recommendations, handles one-tap cart add
2. API gateway (Node.js)	Express server, session manager, rate limiter, input sanitiser	Receives buyer input, manages conversation state, routes to AI or deterministic layer, returns structured response
3. AI layer (Claude)	Anthropic API, L1-L4 system prompt, JSON output schema, retry handler	Intent parsing, progressive narrowing questions, comparison reasoning, confidence scoring, regret prevention, upsell suggestion
4. Deterministic layer	Shopify Admin API client, product fetcher, metafield reader, catalog validator, cart mutator	All Shopify reads and writes. Validates every product_id from LLM output. Executes cart adds. Never delegated to AI.

T1.2 Data flow — a complete request cycle

A complete buyer turn follows this sequence:

Step	What happens
------	--------------

1. Input received	Buyer text or mood tile selection hits the API gateway. Input sanitised. Session state loaded.
2. Catalog fetch (deterministic)	All products with matching occasion metafields fetched from Shopify Admin API via GraphQL. emotion_tags and persona metafields included. Result cached in session for the conversation duration.
3. L1 parse (AI)	Full buyer input + conversation history + catalog summary sent to Claude. System prompt enforces JSON output: {emotion_vectors, occasion_weight, relationship_signal, unstated_constraints, narrowing_question}.
4. L2 narrow (AI)	If confidence < 75%: the narrowing_question from L1 is surfaced to the buyer as the single follow-up. No other output. Conversation continues.
5. L4 recommend (AI)	When confidence threshold reached: Claude returns {product_id, confidence_score, reason, objection, complementary_product_id}.
6. Catalog validation (deterministic)	product_id and complementary_product_id validated against Admin API catalog. If either is invalid: re-prompt with explicit product list. Buyer never sees a hallucinated product.
7. L7 social proof (deterministic)	Persona-matched review metafield fetched for recommended product. Selected by persona match, not random.
8. Response assembly (deterministic)	All outputs assembled into structured response. Confidence score formatted as 'X% match'. Reason rendered in plain English. Objection pre-answered. Complementary product offered after main recommendation.
9. Cart action (deterministic)	On buyer 'add to cart': Shopify Admin API cart mutation called directly. No AI involvement. Confirmation returned synchronously.

T1.3 The AI / deterministic boundary — explicit

AI handles:

Intent parsing (L1), narrowing question generation (L2), product comparison reasoning (L3), confidence scoring (L4), regret objection identification (L8), upsell framing (L9). All outputs are JSON-structured and validated before use.

Deterministic code handles:

All Shopify API calls, all cart operations, all product_id validation, metafield reads, session state management, confidence threshold logic, retry logic, fallback responses. These never touch the LLM.

Why this boundary:

LLMs hallucinate. Any operation that touches real commerce data (inventory, pricing, cart) must be deterministic. The LLM is a reasoning engine, not a data source.

T2. Prompt Architecture

T2.1 System prompt structure

The Claude system prompt has four sections, each enforcing a specific constraint:

Prompt section	What it enforces
Role definition	'You are ShopMind, a gifting decision assistant. You help buyers find the right product through conversation. You never show a list of products. You recommend exactly one product at a time.'
Conversation rules	'Ask exactly one follow-up question per turn. Never ask about budget, occasion, and recipient in the same turn. Build to a recommendation in 2-4 turns. If confidence is below 75% after 4 turns, present the closest match with honest caveats.'
Output schema	'Always respond with valid JSON: {emotion_vectors: [], occasion_weight: 0-1, relationship_signal: string, confidence_score: 0-100, narrowing_question: string null, product_id: string null, reason: string null, objection: string null, complementary_product_id: string null}'
Catalog context	'The available products are: [injected at runtime from Shopify Admin API]. Only recommend product_ids from this list. If no product fits well, set confidence_score below 50 and explain why in reason.'

T2.2 Prompt injection — catalog context

The catalog is injected into the prompt at runtime, not hardcoded. Each product entry includes: product_id, title, price, emotion_tags array, persona array, occasion array, short_description. The prompt injection happens after the deterministic catalog fetch — the LLM never calls the Shopify API directly.

Catalog injection is scoped to the conversation's detected occasion context — if L1 detects a birthday occasion, only birthday-eligible products are injected. This reduces token usage and focuses recommendation reasoning.

T3. Metafield Architecture

T3.1 Custom metafield schema

Three custom metafields are defined for each product in the Shopify Admin. These are the intelligence layer that makes ShopMind's recommendations meaningful rather than keyword-matched.

Metafield key	Type	Example values
shopmind.emotion_tags	JSON array	["warmth", "nostalgia", "comfort", "handmade"]
shopmind.persona	JSON array	["picky-mom", "wellness-seeker", "self-indulgent"]
shopmind.occasion	JSON array	["birthday", "anniversary", "just-because", "diwali"]
shopmind.review_persona	JSON object	{"picky-mom": "My mom is very particular but she loved this...", "wellness-seeker": "I bought this for myself and it became a ritual..."}

shopmind.objection_handler	JSON object	{"price": "This is handmade in small batches — the price reflects...", "shipping": "Arrives within 3 days with gift wrapping included."}
----------------------------	-------------	--

T3.2 Why this architecture enables the 11 logics

Without the metafield schema, ShopMind is just a chatbot that searches product titles. With it: L1 can map 'something warm and personal' directly to emotion_tags. L7 can serve the review written for exactly the buyer's persona. L8 can pre-answer the specific objection most likely for this product. L10 can build a recipient profile from occasion and persona tags accumulated across turns.

T4. Failure Handling — Technical Implementation

Each failure scenario has a defined detection method, response action, and logging behaviour. No failure produces a hallucinated or broken experience for the buyer.

Failure scenario	Detection	Response	Logged
Shopify Admin API timeout (>3s)	try/catch on fetch with 3s timeout	Surface graceful message. Disable cart. No LLM call.	Error + timestamp + endpoint
Shopify API 4xx/5xx response	HTTP status check	Same as timeout. Cache last valid catalog in session for 5 min.	Status code + response body
LLM returns non-JSON	JSON.parse() throws	Retry once with stricter prompt + explicit JSON example. If retry fails: hardcoded clarifying question.	Raw LLM output + turn number
LLM returns invalid product_id	product_id not in session catalog	Re-prompt with explicit product ID list appended to system prompt	Invalid ID + LLM output
LLM returns confidence > 95% on turn 1	Confidence threshold check	Accepted if product_id is valid. High early confidence is allowed if L1 parse is strong.	Flagged for quality review
Confidence stuck below 75% after 4 turns	Turn counter + confidence check	Present closest match with honest framing: 'This is the closest fit — here's why...'	Conversation exported for L11 review
Network disconnect mid-session	WebSocket ping timeout	Session state persisted. Reconnect replays last assistant turn.	Session ID + disconnect timestamp
Buyer input > 500 chars	Input length check at gateway	Truncated to 500 chars before LLM. Buyer notified: 'I'll focus on the most important part of what you said.'	Original input length

T5. Known Limitations and What We Would Improve

Documenting limitations is part of product thinking. A team that claims no limitations has not thought carefully enough.

Limitation	What production would require
Synthetic catalog	40 products with hand-tagged metafields. Production requires: automated metafield tagging pipeline (LLM-assisted first pass + human review), catalog quality checker that flags missing/inconsistent metafields, minimum coverage thresholds before a product enters recommendation pool.
Single session context	L10 recipient profile is session-scoped. Production requires: opt-in persistent profile storage, DPDP-compliant data handling, merchant dashboard showing aggregate persona distribution (never individual data).
No A/B testing on prompt	System prompt is fixed for V1. Production requires: prompt versioning, A/B testing framework on narrowing question strategies, confidence scoring calibration against real purchase outcomes.
English only	L1 parser trained on English input patterns. Production for India requires: Hindi, Tamil, Telugu intent parsing, transliterated Hinglish handling (e.g. 'mummy ke liye kuch warm'), regional occasion awareness.
No real-time inventory	Catalog fetched at conversation start and cached. Production requires: real-time inventory check before cart add, out-of-stock handling that re-runs L2-L4 with updated catalog.
LLM latency	Claude API adds 1-3s per turn. Production requires: streaming response to show buyer the recommendation is building, latency SLA monitoring, fallback to cached recommendations for common intent patterns.

T6. Code Quality and Repository Structure

T6.1 Repository structure

Path	Contents
/src/api/	Express server, route handlers, session manager
/src/ai/	Claude API client, prompt builder, JSON validator, retry handler
/src/shopify/	Admin API client, product fetcher, metafield reader, cart mutator
/src/ui/	React frontend, mood canvas component, chat thread, confidence display
/src/logics/	L1-L11 logic modules — each logic is a separate module with defined input/output contract
/docs/	Product Document (PDF), Technical Document (PDF), DECISION_LOG.md, architecture diagram

/data/	Synthetic product catalog, metafield seed data, persona definitions
README.md	Project name, problem statement, setup instructions, demo video link, contribution note

T6.2 Commit discipline

Commits are atomic and descriptive. Examples: 'add L1 intent parser with JSON schema validation', 'implement catalog validation before LLM product_id is surfaced to buyer', 'add mood canvas mood-to-emotion_tag mapping'. No 'final version', no 'misc fixes' commits. The git history is a readable narrative of build decisions.

The system is only as trustworthy as its deterministic layer.
Every LLM output is validated before a buyer sees it.
That is the architectural principle ShopMind is built on.